Team Name: Wisp
Team Members: Mason Montgomery(MasonMontgomery33) and Steven Rinzler(TidePod06/srinz06)
GitHub Repository: https://github.com/srinz06/P2_GroupProj
Link to Video Demo: https://youtu.be/mS5-JDC8u2A

Problem:

A lot of finance and trading systems today use a priority queue to manage stock data. The most critical part of this process is the data structure used. We are comparing the efficiency of B+ trees and Min Heaps in terms of the inserting, deleting, and searching operations on data from the stock market.

Motivation:

Priority queues are very important for computing systems because they determine how data is prioritized and accessed. They are especially important in applications such as scheduling, event simulation, and financial systems, because the order of processing can significantly impact performance.

Min heaps are commonly used to implement priority queues because of how simple they are and how efficient they can be. They have fast insertion and extraction of the minimum element, making them useful for workloads where only the highest priority element needs to be repeatedly accessed.

B+ trees are widely used in database indexing because they have different advantages than min heaps. In a B+ tree, all the actual data records are stored in the leaf nodes, while internal nodes only store keys for traversing the tree. This means the tree is kept balanced and means many of the operations like inserts, deletes, and searches are O(log n) time.

Features:

Our program contains many different features to test B+ trees and Min Heaps and then compare them. First we build both the structures loaded with 110253 records from our data set and print the time it takes to build for each. We then have 3 different search functions. Simple search(locates a single point of data based on index and date), Range of Entries(Search in order through a range of dates in a given index), and Get Minimum(which accesses the earliest key in each structure). Each search function prints the time for each of the data structures to complete execution. We also have an insert function which inserts a datapoint into both structures, given the index data and all other required information and prints the time it takes to insert and then re-balance the data for both structures. Lastly, we have the delete function which deletes a given node and then rebalances and finally prints the time it took.

Description of Data:

We have 110253 points of stock market data from 14 different indexes from around the world ranging from 1965 to 2021 stored in a csv file. Each point is a day for each index and includes the date, open, high, low, close, adjusted close, and volume for that day. Within each data structure the points are balanced by a priority queue system. Priority is determined by a key which is the concatenation of the index and date in that order. This means the first point or

minimum will be the first date of the first index lexographically, and the last point will be the last date of the last index.

Tools/Languages/APIs/Libraries:

The entire project is coded in C++ and the terminal is used for visuals. We used inclusions from the standard library include <iomanip>, <fstream>, and <sstream> for working with the data from the csv file and <chrono> for timing.

Algorithms Implemented:

B+ Tree

The B+ tree implementation has a lot of core features. The main feature is the **insertion algorithm**, which places new records into leaf nodes while keeping sorted order by key. When a node exceeds its capacity of 64, a split operation is performed, and the middle key is moved up to the parent node to preserve the balanced tree structure.

The **search algorithm** is implemented with a top to bottom traversal starting at the root and going down through the internal nodes by comparing the keys until the correct leaf node is reached, giving O(log n) complexity.

The **deletion algorithm** removes a key from a leaf node and handles the changes if the structure isn't balanced anymore. This could be redistributing keys from sibling nodes or merging nodes to maintain the B+ tree properties.

Additionally, a **range query algorithm** was added using the linked list structure of leaf nodes. Once the starting key is found, then the algorithm just traverses through the leaves to collect all records within the specified range.

A **minimum value retrieval algorithm** was also added and this is done by going through the leftmost path of the tree to the first leaf node.

Min Heap

The Min Heap also has core functions to maintain heap order and support other operations. The main algorithm is **heap insert**, which places a new element at the end of the array and **bubbles/heapify-ups** in order to restore the heap property by repeatedly comparing a node with its parent and swapping when necessary.

The **extracting min algorithm** removes the root element and replaces it with the last element in the heap. The structure is then fixed with a **heapify-down** algorithm, which compares the new root with its children and swaps it downward until the heap property is restored.

A **get minimum algorithm** is implemented in constant time by accessing the root element of the heap.

In addition, a **linear search algorithm** was implemented to locate elements by key, since heap structures do not have ordering like B+ trees. The **deletion algorithm** combines search, removal, and the heapify functions to maintain heap structure after an element is removed.

A **range algorithm** was also added by doing a full traversal of the heap and filtering elements within the specified bounds since heaps do not support ordered traversal.

Additional Data Structures:

On top of the created B+ tree and Min Heap we also used the standard map of a string and a tuple in main.cpp to show stock exchange information such as location, exchange, and currency. Along with this, we used vectors throughout the entire project. B+ trees use them to store keys, child pointers, and records within each node. B+ trees also use a linked list structure on the bottom layer to connect all the data. Lastly the core of a min heap is a vector that uses index calculations for keeping the min heap property.

Responsibility Distribution

Project Setup - Steven Rinzler, B+ tree - Mason Montgomery, Min Heap - Steven Rinzler, Main - both, Report - Both, Video - Steven Rinzler.

Analysis:

Big O

- B+ Tree, n = number of records, h = height of the tree, B = order (max children per node, 64)

search(key): Traverses the tree from root to leaf by comparing the key at each internal node to see of the child pointer is correct, and then repeat this for each level giving O(logn) complexity.

insert(record): Locates the appropriate leaf node using a top to bottom traversal, inserts the record in sorted order within the node's vector, and performs node splitting and moving nodes up a level when the capacity exceeds ORDER, giving O(logn) complexity.

delete_node(key): Finds the target key in a leaf node with a log(n) traversal, removes it from the node's vector, and restores the properties with merging sibling nodes and moving upward propagation, giving O(logn)complexity.

range_get_keys(start, end): Does a log(n) search to locate the starting key in the correct leaf node, and then just traverses the linked list sequentially to get all the records in the range, giving O(logn+k) complexity where k is the number of returned elements.

get_min_val(): Goes from the root node to the first left most leaf node and returns that element, requiring traversal of tree height, meaning it's O(logn) time.

destroy_tree(node): Uses recursion to traverse all internal and leaf nodes using post-order, deallocating memory for each node exactly once, giving O(n) time complexity.

- Min Heap - n = number of records in the heap, h = height of the heap, h=O(logn)

insert(record): Appends the new record to the end of the vector, then restores heap order by repeatedly comparing the element with its parent and swapping the child with the parent to move upward until the property is correct, giving O(log n) time complexity.

extract_min_val(): Removes the root and replaces it with the last element in the vector, reduces the heap size, and performs heapify down by repeatedly swapping with the smaller child until heap order is restored, giving O(log n) complexity.

get_min_val(): Directly accesses the root of the heap at index 0 without traversal or restructuring, and because of how min heaps are structured, this is constant O(1) complexity.

search(key): Does a linear scan through the vector since heap structure does not have ordering, and this compares each element until a match is found or we check the entire heap, giving O(n) time complexity.

delete_node(key): Performs a linear search to locate the element, then swaps it with the last element in the heap, then removes it, and then restores heap property using swaps going up or down, depending on the replaced element's position, giving O(n+logn), which becomes O(n).

range_get_keys(start, end): Traverses the entire heap linearly because the elements are not sorted. It then finds all the nodes that are in that range, giving O(n) time complexity.

size(): Returns the current number of elements stored in the vector, O(1) complexity.

empty_check(): Checks if the vector is empty using a size check, O(1) complexity.

heapify_up(i): Restores heap property by comparing a node with its parent and swapping upward along the height until order is satisfied, giving O(log n)complexity.

heapify_down(i): Restores heap property by repeatedly comparing a node with its left and right children and swapping downward until the correct position is reached, O(log n) complexity.

Timing Comparison - Min Heap is faster for getting the minimum value and building the entire structure, but B+ trees are faster when retrieving values from all across the data board.

Reflection:

Overall, the experience of this project was challenging but rewarding. The best part was getting to finally see and build a real application of what we have been learning in class. Learning the theoreticals and working with small sets of numbers is interesting but you get a whole new view when applying the data structures to a large 100000 point dataset. One of the main challenges we faced was making an essentially identical interface for two entirely different data structures. We had to make sure our steps were aligned the whole way or otherwise we might have to take steps backwards in order to rewrite code to fit a specific format that the main function needed. If

we were to do the project again, we would set up a more detailed understanding of the final interface before starting on the data structures. That way we could have a common goal of functions and actions in mind when setting up the structures. We might also set up more checkpoints to check that the data structures are in line when creating them.

What Each Member learned:

Mason Montgomery - I learned a lot more about B+ trees. Learning their rules and being able to write out the structure after the addition of a few numbers is one thing but making the entire structure from scratch and making sure your program is flawless enough to be able to handle the insertion of 100000 datapoints without an error is a different challenge. It was particularly challenging because of its pointer management throughout which had to be perfect for a working program.

Steven Rinzler - I learned how a Min Heap works in a real program and how to maintain it during insertions and deletions. I also saw that heaps are very fast for getting the minimum value but not good for searches or getting a series of values within a date range. This is because Min Heaps don't have an ordering, and thus it has to traverse each element for a lot of it's operations. This project helped me understand how different data structures perform on larger datasets and when each should be used, as well as the strong suites and weaknesses for each.

References:

Data Set: https://www.kaggle.com/datasets/mattiuzc/stock-exchange-data

B+ Tree Help:

- https://www.youtube.com/watch?v=fVu5b_fCeMY

Min Heap Help:

- https://www.youtube.com/watch?v=UTrr0B4ny9s

CSV File Help:

- https://stackoverflow.com/questions/1120140/how-can-i-read-and-parse-csv-files-in-c
- https://www.youtube.com/watch?v=NFvxA-57LLA